

1. Lab8Q1

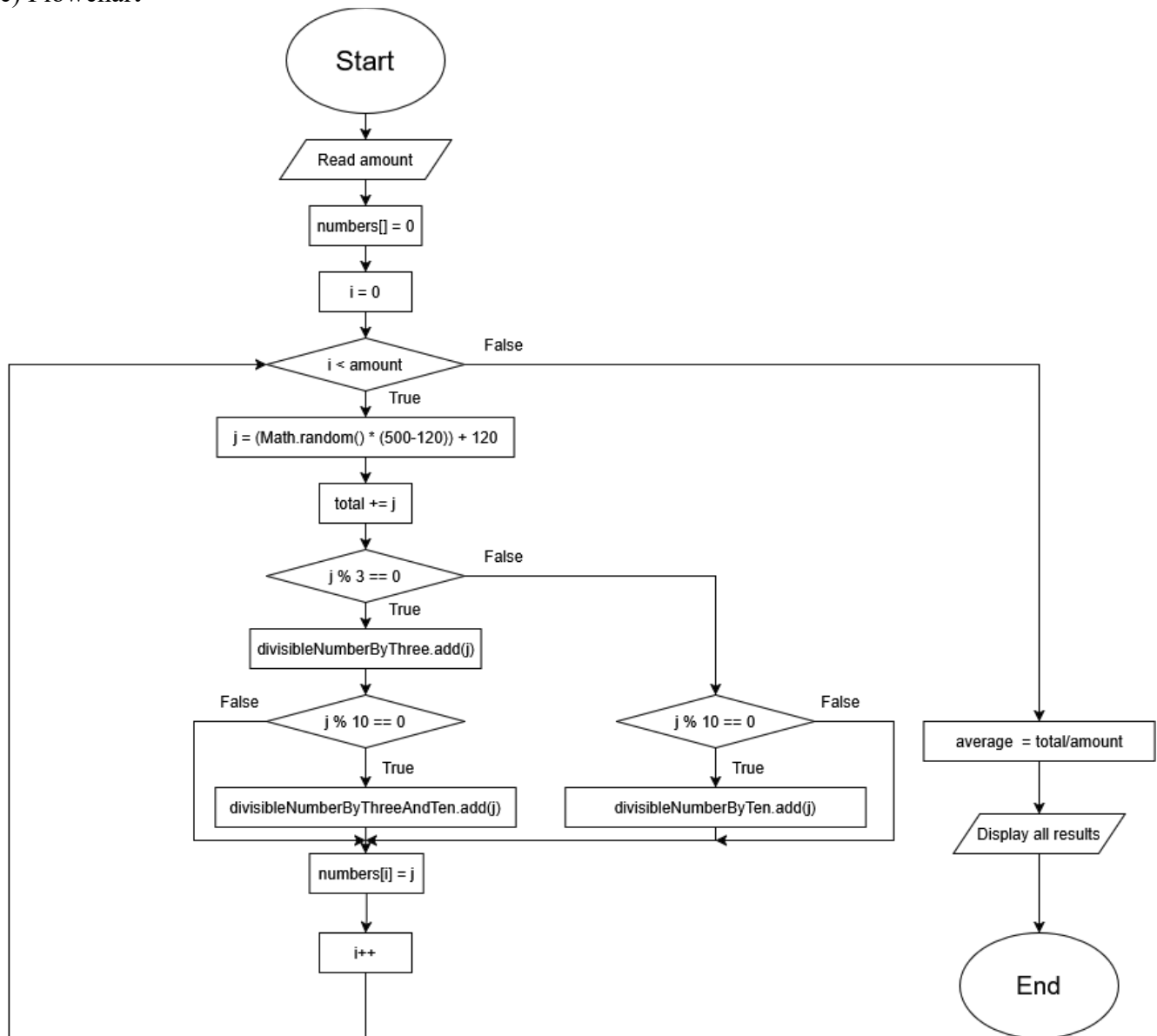
a) Problem Analysis Chart (PAC)

Given Data	Required Results
1. Amount 2. Random generated numbers between 120 to 500	1. divisibleNumberByThree 2. divisibleNumberByTen 3. divisibleNumberByThreeAndTen 4. average
Processing Required	Solution Alternative
1. Read amount 2. Generate amount of numbers between 120 to 500 3. Find numbers that are divisible by three 4. Find numbers that are divisible by ten 5. Find numbers that are divisible by three and ten 6. Display results	1. Define amount as constants. 2. Define amount as input values.

b) Input Process Output (IPO)

Input	Processing	Module Reference	Output
1. Amount 2. Random generated numbers between 120 to 500	1. Read amount 2. Add (Math.random * (500-120)) + 120 into array numbers 3. if(j % 3 == 0) { divisibleNumberByThree.add(j); } if(j % 10 == 0) { divisibleNumberByThreeAndTen.add(j); } } else if(j % 10 == 0){ divisibleNumberByTen.add(j); } 4. total += numbers[i] 5. average = total / amount 6. Display results 7. End	-	1. divisibleNumberByThree 2. divisibleNumberByTen 3. divisibleNumberByThreeAndTen 4. average

c) Flowchart



d) Pseudocode

Start

Read amount

```

for (int i = 0; i < amount; i++){
    j = (Math.random() * (500-120)) + 120
    total += j
    if(j % 3 == 0) {
        divisibleNumberByThree.add(j)
        if(j % 10 == 0) {
            divisibleNumberByThreeAndTen.add(j)
        }
    } else if(j % 10 == 0){
        divisibleNumberByTen.add(j)
    }
}
  
```

```

        numbers[i] = j
    }
    average = total/amount
    Display all results

```

End

e) Java code

```

import java.util.Scanner;
import java.util.InputMismatchException;
import java.util.ArrayList;

public class Lab8Q1{

    public static Scanner scanner = new Scanner(System.in);

    public static void main(String[] args){
        System.out.println("Enter total amount of numbers to be randomly generated between 120 to 500");
        int amount = acceptAmountOfNumber();

        int[] numbers = new int[amount];
        int total = 0;
        ArrayList<Integer> divisibleNumberByThree = new ArrayList<Integer>();
        ArrayList<Integer> divisibleNumberByTen = new ArrayList<Integer>();
        ArrayList<Integer> divisibleNumberByThreeAndTen = new ArrayList<Integer>();

        for (int i = 0; i < amount; i++){
            int j = ((int)(Math.random() * (500-120))) + 120;
            total += j;
            if(j % 3 == 0) {
                divisibleNumberByThree.add(j);
                if(j % 10 == 0) {
                    divisibleNumberByThreeAndTen.add(j);
                }
            } else if(j % 10 == 0){
                divisibleNumberByTen.add(j);
            }
            numbers[i] = j;
        }

        double average = ((double)total)/amount;
        System.out.println("\nAverage : " + average + "\n");

        String percentageThree = String.format("%.2f", (double)divisibleNumberByThree.size()/amount * 100.0);
        String percentageTen = String.format("%.2f", (double)divisibleNumberByTen.size()/amount * 100.0);
        String percentageThreeAndTen = String.format("%.2f", (double)divisibleNumberByThreeAndTen.size()/amount * 100.0);

        System.out.println("There are " + divisibleNumberByThree.size() + " numbers (" + percentageThree + "%) that can be divided by 3 : ");
        System.out.println(divisibleNumberByThree);
        System.out.println();
        System.out.println("There are " + divisibleNumberByTen.size() + " numbers (" + percentageTen + "%) that can be divided by 10 : ");
        System.out.println(divisibleNumberByTen);
        System.out.println();
        System.out.println("There are " + divisibleNumberByThreeAndTen.size() + " numbers (" + percentageThreeAndTen + "%) that can be divided by 3 and 10 : ");
        System.out.println(divisibleNumberByThreeAndTen);
    }

    public static int acceptAmountOfNumber(){
        int input;
        while(true){
            try{
                input = scanner.nextInt();
            } catch (InputMismatchException e){
                System.out.println("Please enter an integer.");
                scanner.nextLine();
                continue;
            }
            if (input < 0){
                System.out.println("Please enter a positive number.");
                scanner.nextLine();
                continue;
            }
            break;
        }
        return input;
    }
}

```

2. Lab8Q2

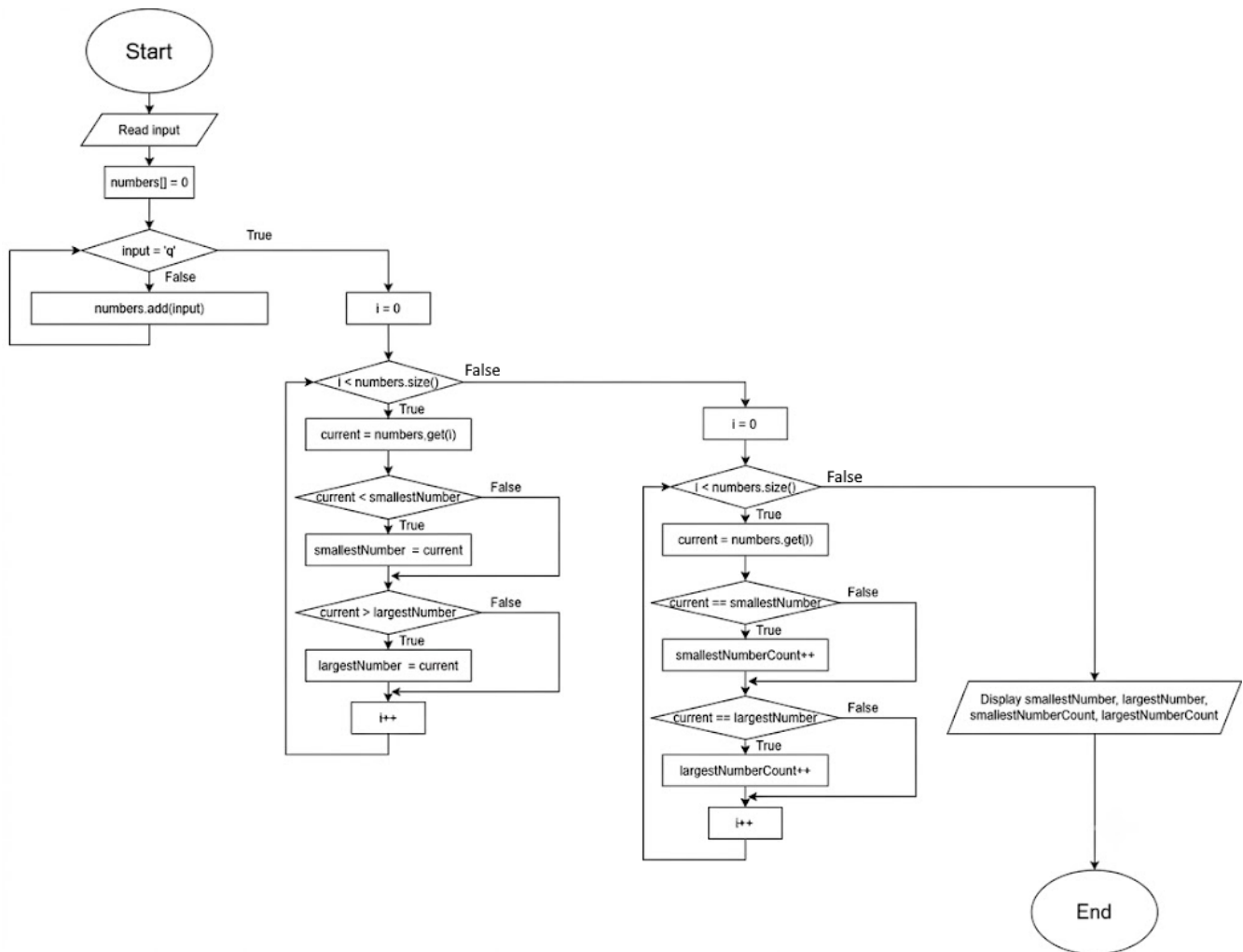
a) Problem Analysis Chart (PAC)

Given Data	Required Results
1. Numbers	1. Smallest number 2. Occurrence of smallest number 3. Largest number 4. Occurrence of largest number
Processing Required	Solution Alternative
1. Read numbers 2. Stop input if user enter 'q' 3. Find smallest number, occurrence of smallest number, largest number and occurrence of largest number 4. Display results	1. Define numbers as constants. 2. Define numbers as input values.

b) Input Process Output (IPO)

Input	Processing	Module Reference	Output
1. Numbers	<pre> 1. Read input 2. if (input == 'q') Jump to 4 else Read input as number 3. Jump to 1 4. for(int i = 1 ; i < numbers.size() ; i++){ current = numbers.get(i) if (current < smallestNumber) smallestNumber = current else if (current > largestNumber) largestNumber = current } 5. for(int i = 0 ; i < numbers.size() ; i++){ current = numbers.get(i) if (current == smallestNumber) smallestNumberCount++ if (current == largestNumber) largestNumberCount++ } 6. Display smallestNumber, largestNumber, smallestNumberCount, largestNumberCount 7. End </pre>	-	1. Smallest number 2. Occurrence of smallest number 3. Largest number 4. Occurrence of largest number

c) Flowchart



d) Pseudocode

Start

Read input

do{

 numbers.add(input)

} while(input != 'q')

for(int i = 1 ; i < numbers.size() ; i++){

 current = numbers.get(i)

 if (current < smallestNumber)

 smallestNumber = current

 else if (current > largestNumber)

 largestNumber = current

}

for(int i = 0 ; i < numbers.size() ; i++){

 current = numbers.get(i)

 if (current == smallestNumber)

 smallestNumberCount++

 if (current == largestNumber)

largestNumberCount++

}

Display smallestNumber, largestNumber, smallestNumberCount, largestNumberCount

End

e) Java code

```
import java.util.Scanner;
import java.lang.NumberFormatException;
import java.util.ArrayList;
// NGO YUK HANG 228991 LAB 8 Q2
public class Lab8Q2{
    public static void main(String[] args){
        System.out.println("This program will find the smallest and largest number entered by user.");
        System.out.println("This program also track the occurrence of the smallest and largest number.");
        System.out.println("Please enter any amount of numbers. Type q to proceed to the calculation.");

        Scanner scanner = new Scanner(System.in);
        String input;
        double number;
        ArrayList<Double> numbers = new ArrayList<Double>();

        while(true){
            input = scanner.nextLine();
            if(input == ""){
                System.out.println("Error. Please enter a valid number.");
                continue;
            }
            if(input.charAt(0) == 'q' || input.charAt(0) == 'Q') {
                if(numbers.size() == 0){
                    System.out.println("Please enter number.");
                    continue;
                }
                break;
            }
            else {
                try{
                    number = Double.parseDouble(input);
                } catch (NumberFormatException e){
                    System.out.println("Error. Please enter a valid number.");
                    //scanner.nextLine();
                    continue;
                }
                numbers.add(number);
                System.out.println("Please input another number or q if finish inputing.");
            }
        }

        double smallestNumber = numbers.get(0);
        double largestNumber = numbers.get(0);
        int smallestNumberCount = 0;
        int largestNumberCount = 0;

        for(int i = 1 ; i < numbers.size() ; i++){
            double current = numbers.get(i);
            if (current < smallestNumber){
                smallestNumber = current;
            } else if (current > largestNumber){
                largestNumber = current;
            }
        }

        for(int i = 0 ; i < numbers.size() ; i++){
            double current = numbers.get(i);
            if (current == smallestNumber) smallestNumberCount++;
            if (current == largestNumber) largestNumberCount++;
        }

        System.out.println("\nYou have entered " + numbers.size() + " numbers.");
        System.out.println("Smallest number is " + smallestNumber + " with " + smallestNumberCount + " times.");
        System.out.println("Largest number is " + largestNumber + " with " + largestNumberCount + " times.");
    }
}
```

3. GradeCalculatorWithMethods (continue from Lab7Q3)

a) Problem Analysis Chart (PAC)

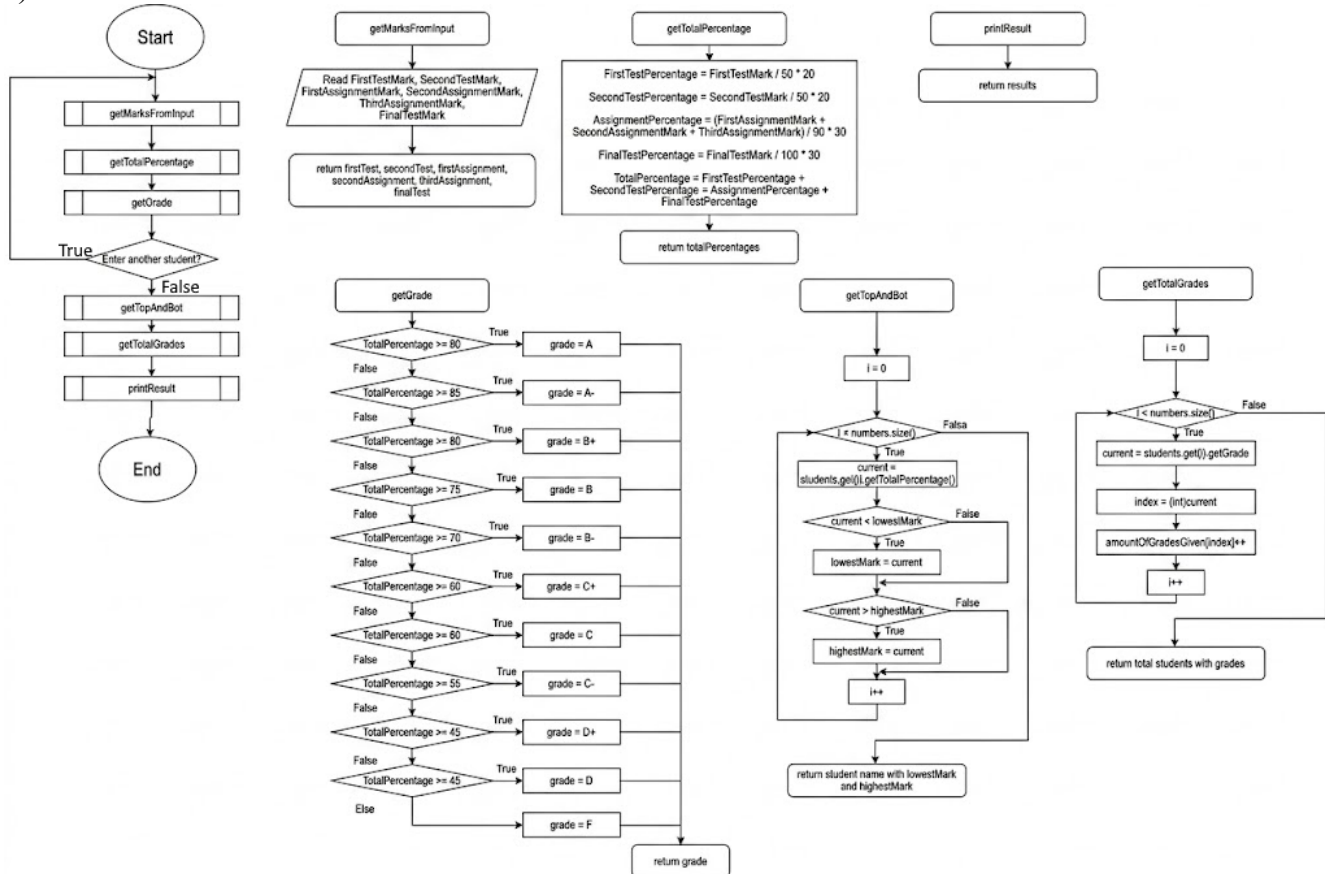
Given Data	Required Results
1. Name 2. Matric number 3. Mark for first test 4. Mark for second test 5. Mark for assignments 6. Mark for final exam	1. All marks in raw mark 2. Total mark in percentage 3. Grade 4. Top scorer name and mark 5. Bot scorer name and mark 6. Total student with grades
Processing Required	Solution Alternative
Read name, matric number, marks $\text{FirstTestInPercentage} = \text{FirstTestInMark} / 50 * 20$ $\text{SecondTestInPercentage} = \text{SecondTestInMark} / 50 * 20$ $\text{AssignmentInPercentage} = (\text{FirstAssignmentInMark} + \text{SecondAssignmentInMark} + \text{ThirdAssignmentInMark}) / 90 * 30$ $\text{FinalTestInPercentage} = \text{FinalTestInMark} / 100 * 30$ $\text{TotalPercentage} = \text{FirstTestInPercentage} + \text{SecondTestInPercentage} + \text{AssignmentInPercentage} + \text{FinalTestInPercentage}$ if(TotalPercentage in some ranges) Grade = A/A-/B+/B/B-/C+/C/C-/D+/D/F if(input != 'q') Read another student data Find top scorer and bot scorer Find total students with grades Display results	1. Define name, matric number and marks for each tests as constants. 2. Define name, matric number and marks for each tests as input values.

b) Input Process Output (IPO)

Input	Processing	Module Reference	Output
1. FirstTest 2. SecondTest 3. First Assignment 4. Second Assignment1.	1. Method to read name 2. Method to read matric number 3. Method to read marks 4. Method to calculate percentage of each test and totalPercentage 5. Method to calculate grade	1. getName 2. getMatric 3. getMarksFromInput 4.	1. All marks in raw mark 2. Total mark in percentage 3. Grade 4. Top scorer

Name 2. Matric number 3. Mark for first test 4. Mark for second test 5. Mark for assignments 6. Mark for final exam 5. Third Assignment 6. FinalTest	6. Ask for another student until user input 'q' 7. Method to find top and bot scorer 8. Method to find total students with grades 9. Method to display all results 10. End	getTotalPercent age 5. getGrade 6. getTopAndBot 7. getTotalGrades 8. printResult	name and mark 5. Bot scorer name and mark 6. Total student with grades
---	--	---	--

c) Flowchart



d) Pseudocode

Main Module

Process getName
 Process getMatric
 Process getMarksFromInput
 Process getTotalPercentage
 Process getGrade

```

    Process getTopAndBot
    Process getTotalGrades
    Process printResult
End

getName Module
    Read name
    Return name
Exit

getMatric Module
    Read matric
    Return matric
Exit

getMarksFromInput Module
    Read FirstTestInMark, SecondTestInMark, FirstAssignmentInMark, SecondAssignmentInMark,
        ThirdAssignmentInMark, FinalTestInMark
    Return FirstTestInMark, SecondTestInMark, FirstAssignmentInMark,
        SecondAssignmentInMark, ThirdAssignmentInMark, FinalTestInMark
Exit

getTotalPercentage Module
    FirstTestInPercentage = FirstTestInMark / 50 * 20
    SecondTestInPercentage = SecondTestInMark / 50 * 20
    AssignmentInPercentage = (FirstAssignmentInMark + SecondAssignmentInMark +
        ThirdAssignmentInMark) / 90 * 30
    FinalTestInPercentage = FinalTestInMark / 100 * 30
    TotalPercentage = FirstTestInPercentage + SecondTestInPercentage + AssignmentInPercentage
        + FinalTestInPercentage
    Return percentages
Exit

getGrade Module
    if (totalPercentage >= 80)
        grade = "A";
    else if (totalPercentage >= 75)
        grade = "A-";
    else if (totalPercentage >= 70)
        grade = "B+";
    else if (totalPercentage >= 65)
        grade = "B";
    else if (totalPercentage >= 60)
        grade = "B-";
    else if (totalPercentage >= 55)
        grade = "C+";
    else if (totalPercentage >= 50)
        grade = "C";
    else if (totalPercentage >= 47)

```

```
        grade = "C-";
    else if (totalPercentage >= 44)
        grade = "D+";
    else if (totalPercentage >= 40)
        grade = "D";
    else
        grade = "F";
    Return grade
```

Exit

getTopAndBot Module

```
    for(int i = 1; i < size;i++){
        current = students.get(i).getTotalPercentage()
        if(current < lowestMark){
            lowestMark = current
        }
        if(current > highestMark){
            highestMark = current
        }
    }
```

Return students name with lowestMark and highestMark

Exit

getTotalGrades Module

```
    for(int i = 1; i < size;i++){
        current = students.get(i).getGrade()
        index = (int)current
        amountGradeScorer[index]++
    }
```

Return total students with grades

Exit

printResult Module

Display results

Exit

e) Java code

GradeCalculatorWithMethodsAndArray.java

```
package Lab8Q3;

import java.util.Scanner;
import java.util.ArrayList;
import java.util.InputMismatchException;

import Lab8Q3.Student;
import Lab8Q3.Task.*;

//NGO YUK HANG 228991 LAB 8 Q3
public class GradeCalculatorWithMethodsAndArray{

    public static Scanner scanner = new Scanner(System.in);
    public static ArrayList<Student> students = new ArrayList<Student>();
    public static ArrayList<Student> topStudents = new ArrayList<Student>();
    public static ArrayList<Student> botStudents = new ArrayList<Student>();

    public static int highestMark = 0;
    public static int lowestMark = 0;
    public static int[] amountGradeScorer = new int[11];

    public static void main(String[] args){
        // Print welcome message
        System.out.println("Welcome to grade calculator!\n");

        // Inputs and calculation
        do{
            Student student = new Student( readName() , readMatric() , readMarks() );
            students.add(student);
            System.out.println(student.printStats());
        } while(hasNextStudent());

        // Display overall stats
        calculateAllStudentsStats();
        displayAllStudentsStats();
    }

    public static String readName(){
        System.out.println("Please enter student name : ");
        return scanner.nextLine();
    }

    public static String readMatric(){
        System.out.println("Please enter matric number : ");
        return scanner.nextLine();
    }
}
```

```

public static int[] readMarks(){
    int[] marks = new int[6];

    System.out.println("\nPlease enter the marks for each tests below.");
    int input;

    for (int i = 0; i < 6; i++){
        while(true){
            System.out.println(Student.markMessages[i]);
            try {
                input = scanner.nextInt();
            } catch (InputMismatchException e){
                System.out.println("Please enter an integer.");
                scanner.nextLine();
                continue;
            }
            if(checkValidMark(input,i)) break;
            else continue;
        }
        marks[i] = input;
    }

    return marks;
}

public static boolean checkValidMark(int input, int index){
    if (input > Student.fullMarks[index] || input < 0){
        System.out.println("The mark should fall between 0 to " + Student.fullMarks[index] + ". Please type again.");
        return false;
    } else return true;
}

public static boolean checkValidMark(int input, int index){
    if (input > Student.fullMarks[index] || input < 0){
        System.out.println("The mark should fall between 0 to " + Student.fullMarks[index] + ". Please type again.");
        return false;
    } else return true;
}

public static void calculateAllStudentsStats(){
    int size = students.size();
    int latestLow = students.get(0).getTotalPercentage();
    int latestHigh = students.get(0).getTotalPercentage();

    for(int j = 1; j < size; j++){
        int current = students.get(j).getTotalPercentage();
        if(current < latestLow){
            latestLow = current;
        }
        if(current > latestHigh){
            latestHigh = current;
        }
    }
    highestMark = latestHigh;
    lowestMark = latestLow;

    for(int i = 0; i < size; i++){
        int temp = students.get(i).getTotalPercentage();
        if(temp == highestMark) topStudents.add(students.get(i));
        if(temp == lowestMark) botStudents.add(students.get(i));
    }

    for(int i = 0; i < size; i++){
        int gradeIndex = students.get(i).getGradeIndex();
        int temp = amountGradeScorer[gradeIndex];
        temp++;
        amountGradeScorer[gradeIndex] = temp;
    }
}

```

```

public static void displayAllStudentsStats(){
    System.out.println("\nThere are " + students.size() + " students.");

    System.out.print("\nTop Scorer (" + highestMark + "%) : ");
    for(int i = 0;i<topStudents.size();i++){
        System.out.print(topStudents.get(i).getName());
        if(i != topStudents.size() - 1) System.out.print(", ");
    }

    System.out.print("\nBot Scorer (" + lowestMark + "%) : ");
    for(int i = 0;i<botStudents.size();i++){
        System.out.print(botStudents.get(i).getName());
        if(i != botStudents.size() - 1) System.out.print(", ");
    }

    System.out.println("\n\nGrades : ");
    System.out.println("A+ : " + amountGradeScorer[0]);
    System.out.println("A : " + amountGradeScorer[1]);
    System.out.println("B+ : " + amountGradeScorer[2]);
    System.out.println("B : " + amountGradeScorer[3]);
    System.out.println("B- : " + amountGradeScorer[4]);
    System.out.println("C+ : " + amountGradeScorer[5]);
    System.out.println("C : " + amountGradeScorer[6]);
    System.out.println("C- : " + amountGradeScorer[7]);
    System.out.println("D+ : " + amountGradeScorer[8]);
    System.out.println("D : " + amountGradeScorer[9]);
    System.out.println("F : " + amountGradeScorer[10]);
}

public static boolean hasNextStudent(){
    System.out.println("Continue to enter another student? (Y/N)");
    char input;
    while(true){
        try{
            input = scanner.next().charAt(0);
        } catch (Exception e){
            continue;
        }
        if(input == 'Y' || input == 'y'){ scanner.nextLine(); return true;}
        else if(input == 'N' || input == 'n'){ scanner.nextLine(); return false;}
        continue;
    }
}
}

```

Student.java

```
package Lab8Q3;

import Lab8Q3.Task.*;

public class Student{
    private String name;
    private String matric;
    private int[] marks = new int[6];
    private int[] percentages = new int[4];
    private int totalPercentage;
    private String grade;

    public static final int[] fullMarks = {50,50,30,30,30,100};
    public static final int[] markToPercentage = {20,20,30,30};

    public static final String[] markMessages = {
        "First test (Full mark is " + fullMarks[0] + ")      : ",
        "Second test (Full mark is " + fullMarks[1] + ")      : ",
        "First assignment (Full mark is " + fullMarks[2] + ")    : ",
        "Second assignment (Full mark is " + fullMarks[3] + ")   : ",
        "Third assignment (Full mark is " + fullMarks[4] + ")    : ",
        "Final test (Full mark is " + fullMarks[5] + ")         : "
    };

    public Student(String name, String matric, int[] marks){
        this.name = name;
        this.matric = matric;
        for(int i = 0; i < 6; i++){
            this.marks[i] = marks[i];
        }
        calculatePercentage();
        calculateGrade();
    }

    private void calculatePercentage(){
        // First test
        percentages[0] = marks[0] * markToPercentage[0] / fullMarks[0];
        // Second test
        percentages[1] = marks[1] * markToPercentage[1] / fullMarks[1];
        // Assignments
        percentages[2] = (marks[2] + marks[3] + marks[4]) * markToPercentage[2] / (fullMarks[2] + fullMarks[3] + fullMarks[4]);
        // Final test
        percentages[3] = marks[5] * markToPercentage[3] / fullMarks[5];

        // Calculate total percentage
        totalPercentage = percentages[0] + percentages[1] + percentages[2] + percentages[3];
    }
}
```

```

private void calculateGrade(){
    if (totalPercentage ≥ 80)
        grade = "A";
    else if (totalPercentage ≥ 75)
        grade = "A-";
    else if (totalPercentage ≥ 70)
        grade = "B+";
    else if (totalPercentage ≥ 65)
        grade = "B";
    else if (totalPercentage ≥ 60)
        grade = "B-";
    else if (totalPercentage ≥ 55)
        grade = "C+";
    else if (totalPercentage ≥ 50)
        grade = "C";
    else if (totalPercentage ≥ 47)
        grade = "C-";
    else if (totalPercentage ≥ 44)
        grade = "D+";
    else if (totalPercentage ≥ 40)
        grade = "D";
    else
        grade = "F";
}

public String printStats(){
    String stats = "\n-----";
    stats += "\nName : " + name;
    stats += "\nMatric : " + matric;
    stats += "\nResult : \n";

    for (int i = 0;i<6;i++){
        stats += markMessages[i] + marks[i] + "\n";
    }

    stats += "Total percentage : " + totalPercentage + "%\n";
    stats += "Grade : " + grade + "\n";
    stats += "-----\n";

    return stats;
}

```

```

    public String getName(){
        return name;
    }

    public String getMatric(){
        return matric;
    }

    public int getMark(Task.Mark index){
        return marks[index.ordinal()];
    }

    public int getPercentage(Task.Percentage index){
        return percentages[index.ordinal()];
    }

    public int getTotalPercentage(){
        return totalPercentage;
    }

    public String getGrade(){
        return grade;
    }

    public int getGradeIndex(){
        switch(grade){
            case "A" : return 0;
            case "A-" : return 1;
            case "B+" : return 2;
            case "B" : return 3;
            case "B-" : return 4;
            case "C+" : return 5;
            case "C" : return 6;
            case "C-" : return 7;
            case "D+" : return 8;
            case "D" : return 9;
            case "F" : return 10;
        }
        return -1;
    }
}

```

Task.java

```

package Lab8Q3;

public class Task{
    public enum Mark{
        TEST_1,
        TEST_2,
        ASSIGNMENT_1,
        ASSIGNMENT_2,
        ASSIGNMENT_3,
        TEST_FINAL
    }
    public enum Percentage{
        TEST_1,
        TEST_2,
        ASSIGNMENTS,
        TEST_FINAL
    }
}

```

4. Lab8Q4

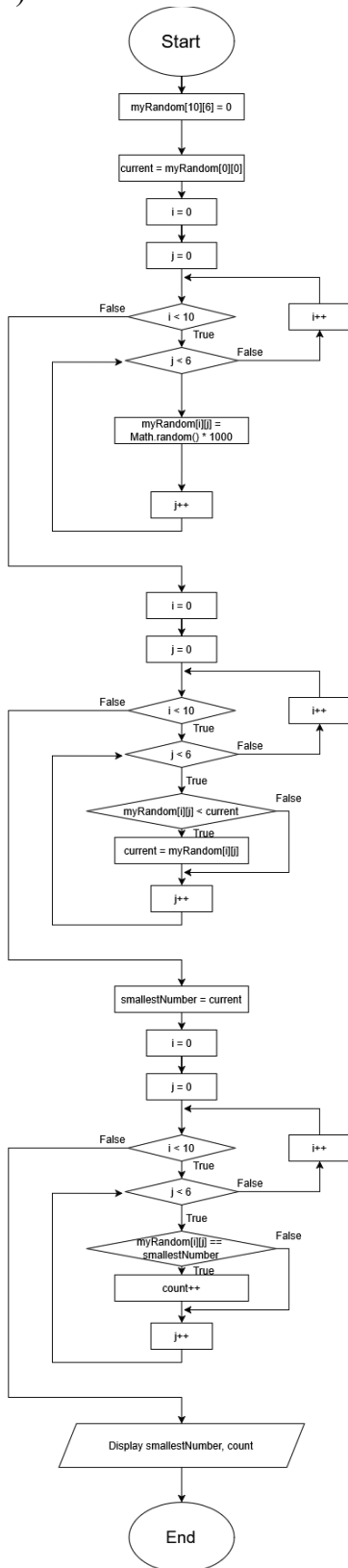
a) Problem Analysis Chart (PAC)

Given Data	Required Results
1. 10 by 6 array with random generated numbers between 0 to 1000	1. Smallest number 2. Occurrence of smallest number
Processing Required	Solution Alternative
1. Prepare array as myRandom 2. Find smallestNumber 3. Find count 4. Display smallestNumber 5. End	1. Define myRandom as constants.

b) Input Process Output (IPO)

Input	Processing	Module Reference	Output
1. 10 by 6 array with random generated numbers between 0 to 1000	1. int[][] myRandom = new int[10][6] 2. for loop myRandom[i][j] = Math.random() * 1000 3. for loop find smallestNumber 4. for loop find count 5. Display smallestNumber, count 6. End	-	1. Smallest number 2. Occurrence of smallest number

c) Flowchart



d) Pseudocode

Start

```
int[][] myRandom = new int[10][6]
for(int i = 0 ; i < 10 ; i++){
    for(int j = 0; j < 6 ; j++){
        myRandom[i][j] = Math.random() * 1000
    }
}

int current = myRandom[0][0];
for(int i = 0 ; i < 10 ; i++){
    for(int j = 0; j < 6 ; j++){
        if(myRandom[i][j] < current){
            current = myRandom[i][j]
        }
    }
}
smallestNumber = current

for(int i = 0 ; i < 10 ; i++){
    for(int j = 0; j < 6 ; j++){
        if(myRandom[i][j] == smallestNumber){
            count++
        }
    }
}
```

Display smallestNumber, count

End

e) Java code

```
public class Lab8Q4{
// NGO YUK HANG 228991 LAB 8 Q4
    public static void main(String[] args){
        int[][] myRandom = new int[10][6];
        for(int i = 0 ; i < 10 ; i++){
            for(int j = 0; j < 6 ; j++){
                myRandom[i][j] = (int)(Math.random() * 1000);
            }
        }

        int smallestNumber;
        int current = myRandom[0][0];
        for(int i = 0 ; i < 10 ; i++){
            for(int j = 0; j < 6 ; j++){
                if(myRandom[i][j] < current){
                    current = myRandom[i][j];
                }
            }
        }
        smallestNumber = current;

        int count = 0;
        for(int i = 0 ; i < 10 ; i++){
            for(int j = 0; j < 6 ; j++){
                if(myRandom[i][j] == smallestNumber){
                    count++;
                }
            }
        }

        System.out.println("The smallest number is " + smallestNumber + " with " + count + " occurrence.");
    }
}
```

5. Lab8Q5

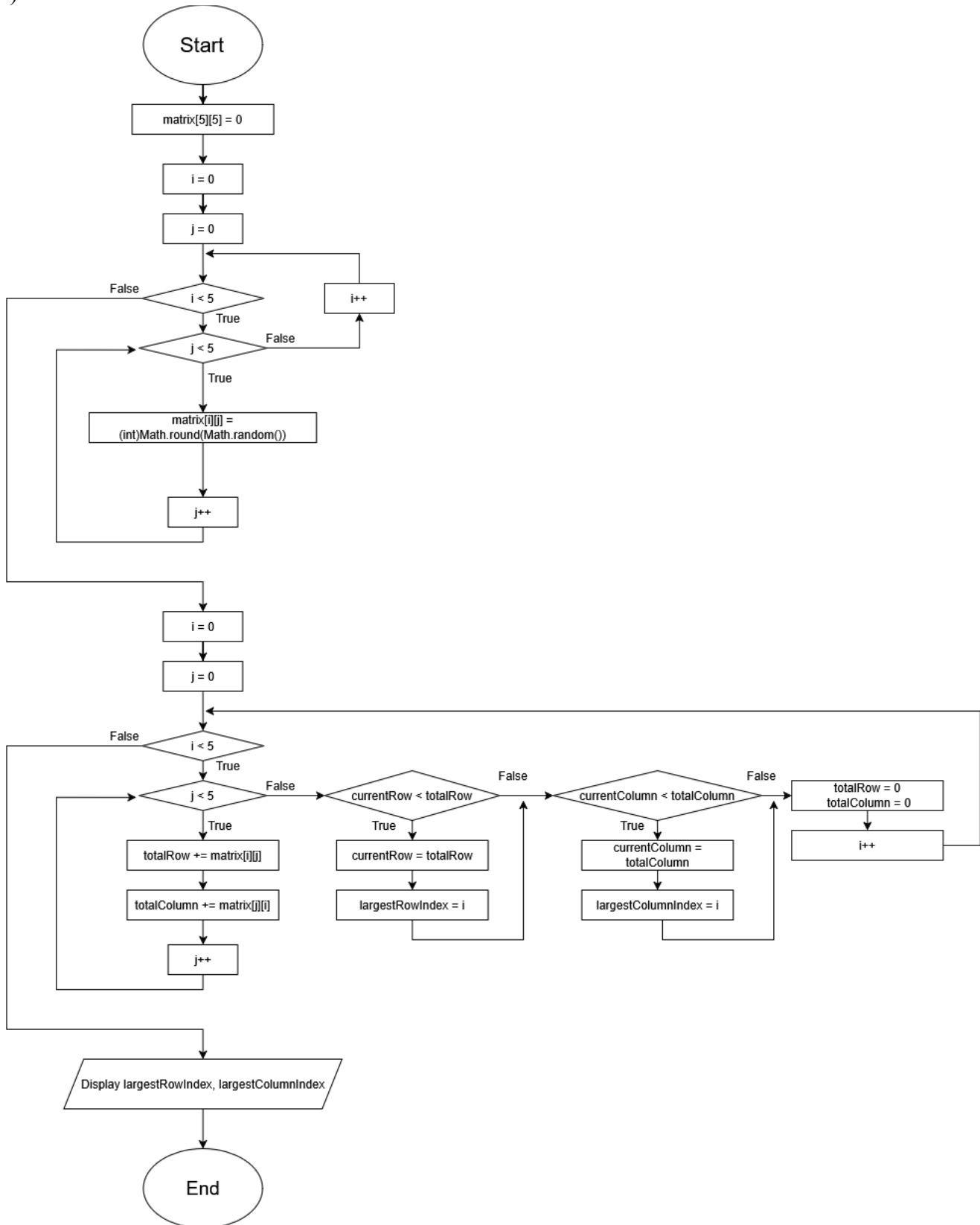
a) Problem Analysis Chart (PAC)

Given Data	Required Results
1. 5 by 5 array filled with 0 or 1	1. Largest row index 2. Largest column index
Processing Required	Solution Alternative
1. Prepare array as matrix 2. Find largestRowIndex, largestColumnIndex 3. Display matrix 4. Display largestRowIndex, largestColumnIndex 5. End	1. Define matrix as constants.

b) Input Process Output (IPO)

Input	Processing	Module Reference	Output
1. 5 by 5 array filled with 0 or 1	1. int[][] matrix = new int[5][5] 2. for loop matrix[i][j] = Math.round(Math.random()) 3. for loop find largestRowIndex 4. for loop find largestColumnIndex 5. Display largestRowIndex, largestColumnIndex 6. End	-	1. Largest row index 2. Largest column index

c) Flowchart



d) Pseudocode

Start

```
int[][] matrix = new int[5][5]
for(int i = 0 ; i < 5 ; i++){
    for(int j = 0; j < 5 ; j++){
        matrix[i][j] = (int)Math.round(Math.random())
    }
}

for(int i = 0;i<5;i++){
    for(int j = 0;j<5;j++){
        totalRow += matrix[i][j];
        totalColumn += matrix[j][i];
    }
    if(currentRow < totalRow){
        currentRow = totalRow;
        largestRowIndex = i;
    }
    if(currentColumn < totalColumn){
        currentColumn = totalColumn;
        largestColumnIndex = i;
    }
    totalRow = 0;
    totalColumn = 0;
}
```

Display largestRowIndex, largestColumnIndex

End

e) Java code

```
public class Lab8Q5{
// NGO YUK HANG 228991 LAB 8 Q5
    public static void main(String[] args){
        int[][] matrix = new int[5][5];
        for(int i = 0;i<5;i++){
            for(int j=0;j<5;j++){
                matrix[i][j] = (int)Math.round(Math.random());
            }
        }

        int largestRowIndex = 0;
        int largestColumnIndex = 0;
        int currentRow = 0;
        int currentColumn = 0;
        int totalRow = 0;
        int totalColumn = 0;
        for(int i = 0;i<5;i++){
            for(int j = 0;j<5;j++){
                totalRow += matrix[i][j];
                totalColumn += matrix[j][i];
            }
            if(currentRow < totalRow){
                currentRow = totalRow;
                largestRowIndex = i;
            }
            if(currentColumn < totalColumn){
                currentColumn = totalColumn;
                largestColumnIndex = i;
            }
            totalRow = 0;
            totalColumn = 0;
        }
        for(int i = 0;i<5;i++){
            if(i == largestRowIndex) System.out.print(">");
            else System.out.print(" ");

            for(int j = 0;j<5;j++){
                System.out.print(matrix[i][j]);
            }
            System.out.print("\n");
        }
        String space = " ";
        for(int k = 0;k<largestColumnIndex;k++){
            space += " ";
        }
        System.out.println(space + "^");

        System.out.println("The largest row index : " + largestRowIndex);
        System.out.println("The largest column index : " + largestColumnIndex);
    }
}
```